Memory Management (new/delete)

In C++, memory is divided into two primary regions: the **Stack** and the **Heap**.

1. The Stack (Automatic Memory)

By default, when you create a normal variable inside a function it is placed on the Stack **(**e.g., **int x = 5** or **Car myCar).**

How it works: The Stack is extremely fast and strictly organized (Last-In, First-Out).

The Catch: The memory is entirely managed for you.

- When a function finishes, all the local variables inside it are automatically destroyed and cleared.
- You cannot create massive amounts of data on the Stack, or you will cause a **"Stack Overflow."**

2. The Heap (Dynamic Memory) & The new Operator:

If you need an object to survive after a function ends, or if you need to allocate a massive amount of memory (like a huge array for a data structure) where you don't know the size until the program is actually running, you must use the **Heap**.

To put something on the Heap, you use the **"new"** keyword.

What new does:

1. It finds a block of free memory on the Heap large enough for your object.
2. It calls the object's constructor to initialize it.
3. It **returns a pointer** (the exact memory address) to where that object lives.

```
// Allocating an integer, an Object, an array of integers on the Heap
int *myNumber = new int(42);
Car *myCar = new Car("Toyota");
int *myGrid = new int[100];
```

3. The delete Operator (Taking out the trash)

If you put something on the Heap, it stays there forever until you explicitly destroy it using delete.

What delete does:

1. It calls the object's destructor (giving it a chance to clean up its internal data).
2. It frees the memory back to the system so it can be used again.

```
// Deleting a single object
delete myNumber;
delete myCar;

// CRITICAL: Deleting an array requires brackets []
delete[] myGrid;
```

The Two Greatest Dangers of Manual Memory

A. Memory Leaks: happens when you use **"new"** to allocate memory, but you forget (or fail) to call **"delete".** If you write a loop that keeps using new without delete, your program will slowly eat up all the RAM on your computer until the operating system violently kills the program.

```
void badFunction() {
  Car *myCar = new Car("Toyota");
  // We finish the function, but forgot to call 'delete lostDog;'
  // 'Rex' is now on the Heap forever, taking up space.
}
```

B. Dangling Pointers: This happens when you delete an object, but you still have a pointer trying to look at that memory address. It is like having the address to a house that has been demolished. If you try to use a dangling pointer, your program will usually crash instantly (Segmentation Fault).

```
Car *myCar = new Car();
delete myCar; // The memory is freed. myCar is now a dangling pointer.

myCar->start(); // If you do this, the program will crash:

myCar = nullptr; // Best practice: Set pointers to nullptr after deleting them
```

The Modern C++ uses Smart Pointers (See Smart Pointer Topic)

Stack Overflow

Occurs when a computer program runs out of memory in its **call stack**. The most frequent culprit is **infinite recursion**.

Think of it like a physical stack of trays. Every time a function is called, a new tray is added to the top. When it ends, the tray is removed. If you keep adding trays without removing any, it eventually hits the ceiling and collapses.

```
void crash() { crash(); // The function calls itself forever }
```

Key Technical Details

- **The Stack:** A small, fast region of RAM reserved for thread execution.
- **Memory Limit:** The Stack has a strict, fixed size set by the operating system or the compiler.
- **The Result:** When the stack pointer exceeds the stack bound, the program immediately crashes with a Segmentation Fault or a StackOverflowError.